

Code Complexity Metrics for AI-Assisted Development: A Survey and Comparative Tool Analysis

Brandon Arrendondo

brandon.arrendondo@bissell.com

June 26, 2026

Abstract

Code complexity metrics have guided software quality practice for nearly five decades, from McCabe’s cyclomatic complexity [?] in 1976 through Cognitive Complexity [?] in 2018. These metrics were designed to model *human* cognitive load. The rise of AI-assisted development introduces a different cost model: the effort and token budget an LLM requires to safely modify a function. This survey reviews 10 established complexity metrics, evaluates 8 open-source analysis tools across 5 dimensions (metric coverage, language support, CI integration, output formats, and speed), and characterizes the gap between existing metrics and the requirements of AI-cost modeling. No surveyed tool computes metrics oriented toward LLM modification cost, and only two support the multi-language, CI-native deployment model needed for modern polyglot codebases. We describe the design requirements for an AI-cost-aware complexity tool and position knots [?] as the first open-source tool to address this gap.

1 Introduction

Software complexity metrics serve two related purposes: they identify functions that are difficult to understand, and they provide actionable signals for refactoring. Both purposes assume a model of the reader. McCabe’s cyclomatic complexity models the *structural* cost of testing all paths through a function. Cognitive Complexity models the *reading* difficulty a

human programmer experiences. Halstead’s software science models *vocabulary and volume* as proxies for mental effort.

None of these metrics was designed for the reader a modern development team uses most: an LLM assistant such as Claude, GPT-4, or Gemini. LLM-assisted development has a fundamentally different cost model. The cost of asking an LLM to modify a function is not measured in programmer minutes; it is measured in:

1. **Reasoning effort** — the number of branch paths, nesting levels, and state variables the model must track to generate a correct edit.
2. **Context tokens** — the tokens consumed loading the function itself and all external symbols it references before reasoning can begin.

Traditional complexity metrics partially correlate with these costs but were not calibrated against them. A 300-line function with simple linear structure scores high on SLOC but low on McCabe; it is cheap to reason about (few branches) but expensive to tokenize (many lines). A 20-line function with 15 external calls scores low on all traditional metrics but has high context pressure: the model must resolve 15 dependency chains before acting.

This survey makes three contributions:

1. A structured review of 10 established complexity metrics, organized by what aspect of complexity each models.

Metric	Category	Year	Models	
McCabe CC	Structural	1976	Test paths	McCabe was designed to measure the minimum number of test cases needed to achieve branch coverage. It is the most widely deployed complexity metric: tools, linters, and style guides from NASA’s JPL [?] to MISRA C [?] cite McCabe thresholds (typically 10–15 as a warning threshold, 25 as a hard limit).
NPath	Structural	1988	Execution paths	
ABC	Structural	1997	Assignment/Branch/Condition	
Cognitive CC	Structural	2018	Reading difficulty	
Nesting Depth	Structural	—	Stack depth	
SLOC	Size	—	Length	Limitation for AI cost: McCabe counts branches but not nesting depth. A deeply nested function with 10 branches scores identically to a flat function with 10 independent checks. LLMs penalize nesting more heavily than flatness: a nested structure forces the model to maintain a context stack; a flat structure does not.
Token Count	Size	—	Lexical volume	
Halstead Vol	Vocabulary	1977	Program vocabulary	
Halstead Dif	Vocabulary	1977	Implementation difficulty	
AIRD	AI-cost	2025	LLM reasoning effort	
AICP	AI-cost	2025	LLM context tokens	

Table 1: Complexity metrics surveyed, by category.

2. A comparative evaluation of 8 open-source analysis tools on 5 dimensions relevant to CI-native deployment.
3. A characterization of the gap between existing tools and AI-cost-aware complexity analysis, motivating the AIRD and AICP metrics described in the companion paper [?].

The rest of this paper is organized as follows. ?? surveys established complexity metrics. ?? evaluates open-source tools. ?? characterizes the gap. ?? covers related work. ?? concludes.

2 Complexity Metrics

We organize complexity metrics into four categories: structural, size-based, vocabulary-based, and AI-cost-oriented. ?? summarizes key properties.

2.1 Structural Metrics

2.1.1 McCabe Cyclomatic Complexity

McCabe [?] defines cyclomatic complexity as $V(G) = e - n + 2p$ for a control-flow graph with e edges, n nodes, and p connected components. In practice this reduces to counting decision points plus one: each `if`, `while`, `for`, `case`, `||`, and `&&` contributes one unit.

2.1.2 NPath Complexity

Nejmeh [?] defines NPath as the number of acyclic execution paths through a function. For a function with n independent branches, $NPath = 2^n$; two sequential `if` statements produce $NPath = 4$. NPath grows exponentially and is considered complementary to McCabe, which grows linearly.

Limitation: NPath is rarely used in practice because exponential growth makes thresholds hard to set and communicate. The lizard tool computes it but does not recommend thresholds.

2.1.3 ABC Complexity

Fitzpatrick [?] defines ABC complexity as a three-component vector (A, B, C) where A counts assignment operations, B counts branch operations (function calls and method dispatches), and C counts condition checks. The scalar magnitude $\|ABC\| = \sqrt{A^2 + B^2 + C^2}$ is reported as the complexity score.

ABC was designed for Ruby code quality and is the basis of Rails’ Flog tool. It is useful when branch count and assignment density are both relevant, but it conflates two orthogonal signals (function call density and branching) into one scalar.

2.1.4 Cognitive Complexity

Campbell [?] introduced Cognitive Complexity in 2018 as a direct response to the limitations of McCabe. The key differences are:

- Nested structures incur a nesting penalty — each level of nesting adds to the cost of the enclosing control structure.
- Sequences of `else if` are charged once, not once per branch.
- `switch` is a single increment regardless of arm count.
- Recursive calls and co-routines incur a flat increment.

Cognitive Complexity has been validated empirically: Campbell cites SonarSource data showing higher correlation with developer-reported difficulty than McCabe. Mozilla’s `rust-code-analysis` computes Cognitive Complexity for Rust, C, and Python; knots was validated against it at $1.004\times$ mean ratio across 11,365 Rust functions.

Limitation for AI cost: Cognitive Complexity models *human* structural comprehension. It does not account for external dependency breadth (AICP) or state access patterns (state coupling).

2.1.5 Nesting Depth

Maximum nesting depth is a direct complement to Cognitive Complexity. While Cognitive Complexity accumulates nesting penalties, nesting depth reports the worst-case stack depth the reader must maintain. A function with a single 6-deep nesting chain scores low on McCabe and moderate on Cognitive Complexity, but the nesting depth signal is unambiguous.

The JPL coding standard [?] limits nesting to 2 levels; MISRA C recommends 4. Both reflect the same intuition: deep nesting imposes disproportionate maintenance cost.

2.2 Size-Based Metrics

2.2.1 SLOC

Source lines of code (SLOC) is the oldest and most broadly understood complexity proxy. It does not measure anything structural about the code; it measures how long the reader must engage with it. Long functions are correlated with high complexity, but

the correlation is loose: a 300-line `switch` statement with 100 cases is long but structurally simple.

Different tools define SLOC differently. Physical SLOC counts every non-blank line; logical SLOC counts statements. Most modern tools use physical SLOC minus blank lines and comment-only lines. knots follows this definition, verified against `cloc` counts.

2.2.2 Token Count

Lizard and some research tools report token count as a proxy for function size. Token count is more stable than SLOC across formatting styles (brace placement, line wrapping) but is rarely used as a primary threshold metric in practice.

2.3 Vocabulary-Based Metrics

2.3.1 Halstead Software Science

Halstead [?] proposed a family of metrics based on operator and operand vocabulary:

$$\text{Volume} = N \cdot \log_2 \eta \quad (1)$$

$$\text{Difficulty} = \frac{\eta_1}{2} \cdot \frac{N_2}{\eta_2} \quad (2)$$

$$\text{Effort} = \text{Difficulty} \times \text{Volume} \quad (3)$$

where η_1 is the count of unique operators, η_2 unique operands, N_1 total operator occurrences, N_2 total operand occurrences, and $\eta = \eta_1 + \eta_2$.

Halstead metrics are computed by several tools (radon for Python, `rust-code-analysis` for Rust) but have largely fallen out of favor in applied software engineering. Shepperd [?] found that Halstead’s predictions were no better than SLOC alone for effort estimation. They remain in use in research as one component of composite metrics (see maintainability index, below).

2.4 Composite Metrics

2.4.1 Maintainability Index

The Maintainability Index (MI) [?] combines Halstead volume, McCabe cyclomatic complexity, and

SLOC into a single score:

$$\begin{aligned} \text{MI} = & 171 - 5.2 \cdot \ln(\text{HV}) \\ & - 0.23 \cdot V(G) - 16.2 \cdot \ln(\text{SLOC}) \end{aligned} \quad (4)$$

MI is normalized to a 0–100 scale in some tools (Visual Studio uses a variant). It is the most widespread composite metric but suffers from the compound weaknesses of its components: Halstead volume’s weak predictive value drags on the formula, and the logarithm compression makes the score insensitive to large changes at the high end.

2.5 AI-Cost Metrics

AIRD and AICP are described in detail in the companion paper [?]. We include them in this taxonomy because they represent a qualitatively new category: metrics calibrated to model *LLM cost* rather than human cognitive load. The key design choices — separating reasoning effort (AIRD) from context pressure (AICP), calibrating ceiling values against empirical p99 distributions, and incorporating state coupling as a dampener for mechanical splits — have no precedent in the prior metrics literature.

3 Tool Evaluation

We evaluate 8 open-source tools against 5 dimensions: metric coverage, language support, CI integration ergonomics, structured output formats, and analysis speed. ?? gives the overview; the subsections provide detail on notable characteristics.

3.1 pmccabe

pmccabe [?] is the canonical McCabe implementation for C and C++. It has been in continuous use since the 1990s and produces output that knots was validated against (100% match across 32,205 functions). Its scope is deliberately narrow: it computes and reports McCabe complexity, nothing else. It has no structured output, no threshold configuration, and no CI integration beyond shell scripting.

pmccabe remains the appropriate tool when the single concern is McCabe compliance in a C/C++ shop, but it does not scale to the multi-metric, multi-language, CI-native requirements of modern development.

3.2 lizard

Lizard [?] is a Python-based multi-language tool that computes McCabe CC, SLOC, parameter count, and token count for 27 programming languages. It is the most broadly deployed open-source complexity scanner: CI configurations referencing lizard appear in tens of thousands of public repositories.

Lizard exits non-zero on threshold violations and produces JSON output, making it the most CI-ready tool in the survey after knots. Its primary limitations are: it does not compute Cognitive Complexity; its SLOC counting differs from standard definitions for several languages; and it has no pre-commit framework integration, requiring manual CI configuration.

3.3 radon

Radon [?] is a Python-only tool that computes the broadest metric set of any single-language tool: McCabe CC, Halstead volume/difficulty/effort, Maintainability Index, and SLOC. It is the standard choice for Python teams that want Halstead metrics without a heavyweight tool. Its JSON output and threshold-based exit codes make it CI-compatible.

The Python-only scope is radon’s primary limitation. Teams working in polyglot codebases cannot use radon as a single-pane-of-glass solution.

3.4 rust-code-analysis

Mozilla’s rust-code-analysis (RCA) [?] computes the most comprehensive metric set of any open-source tool: McCabe, Cognitive Complexity, Halstead family, SLOC, comment density, function count, and several structural metrics. It supports Rust, C/C++, Python, and JavaScript via tree-sitter.

RCA is a research artifact — it was designed for Mozilla’s internal code quality research and as a validated reference implementation for Cognitive Com-

Tool	Lang(s)	Metrics
pmccabe [?]	C, C++	CC
lizard [?]	27 langs	CC, SLOC, params, tokens
radon [?]	Python only	CC, Halstead, MI, SLOC
rust-code-analysis [?]	Rust, C, C++, JS	CC, Cognitive, Halstead, SLOC, many more
tokei [?]	150+ langs	SLOC only
SonarQube [?]	30+ langs	CC, Cognitive, SLOC, duplications, smells
understand [?]	20+ langs	CC, Halstead, many more
knots [?]	C, C++, Rust, Py, JS	CC, Cognitive, Nesting, SLOC, ABC, Returns, TestScore, ExternalCalls, AIRD, AICP, StateCou

Table 2: Open-source complexity tools surveyed. “CI exit” = tool exits non-zero on threshold violation. “pre-commit” = native pre-commit framework integration. “Speed” = qualitative assessment on a 100K-LOC codebase. CC = cyclomatic complexity.

plexity. It produces JSON output but does not provide threshold-based exit codes, making CI integration purely manual. There is no pre-commit hook and no documented threshold recommendations. The tool is not actively maintained for deployment use cases.

3.5 tokei

Tokei [?] supports over 150 languages and reports SLOC, blank lines, comment lines, and total lines per file and language. It does not compute any structural metric. Its value is as a fast, reliable SLOC counter across large polyglot codebases, not as a complexity analyzer. We include it because many teams use tokei alongside a structural analyzer.

3.6 SonarQube

SonarQube [?] is a server-based platform that computes McCabe CC, Cognitive Complexity, SLOC, code duplication, and code smells across 30+ languages. Its Cognitive Complexity implementation matches Campbell’s specification. SonarQube is the most full-featured tool in the survey, but its server deployment model, licensing structure (many features require SonarQube Developer Edition or higher), and analysis latency (full project scans take minutes to hours) make it unsuitable for pre-commit or per-function CI gate use cases.

SonarCloud (the hosted SaaS variant) has better CI integration via GitHub Actions / GitLab CI, but

shares the same latency and cost characteristics.

3.7 Understand

Scientific Toolkit’s Understand [?] is a commercial static analysis IDE that computes over 40 complexity metrics including McCabe, Halstead, and several call-graph-based metrics. It is the most comprehensive tool in the survey in raw metric count, but it is commercial, requires manual invocation through a GUI or script, and does not integrate into pre-commit or lightweight CI workflows.

3.8 knots

Knots [?] is a Rust-based, multi-language complexity analyzer designed for CI-native deployment. It computes 13 metrics per function: McCabe CC, Cognitive Complexity, nesting depth, SLOC, ABC magnitude, return count, test scoring (5 sub-dimensions), external call count, AIRD, AICP, and state coupling.

Knots was validated against reference implementations: 100% McCabe match against pmccabe across 32,205 functions; 1.004× mean Cognitive match against rust-code-analysis across 11,365 Rust functions. It supports C, C++, Rust, Python, and JavaScript via tree-sitter, exits non-zero on configurable threshold violations, produces JSON/NDJSON/SARIF/CSV output, and ships a pre-commit framework hook.

The metrics not found in any other surveyed tool are AIRD, AICP, and state coupling, which are cov-

ered in the companion paper [?].

4 The AI-Cost Gap

4.1 What Existing Metrics Miss

We identify three gaps between existing complexity metrics and the requirements of AI-cost modeling.

Context pressure is unmeasured. No surveyed metric captures the number of external symbols a function requires resolved before it can be understood. A function that calls 15 external APIs forces an LLM to expand 15 dependency chains before generating an edit; a 300-line linear function forces tokenization of 300 lines. These are distinct costs. External call count captures the former; SLOC captures the latter. No existing composite metric combines them in a way calibrated to LLM token budgets.

State access patterns are unmeasured. Traditional metrics do not distinguish between a function’s explicit interface (parameter count) and its actual state dependencies (fields accessed from a passed object or `self`). The state coupling term in AIRD was motivated by a specific failure mode: a god-struct anti-pattern where struct-wrapping parameters hides coupling from all arity-based metrics without actually reducing the state the function touches (??).

Reasoning vs. context are conflated. Existing composite metrics (MI, Halstead Effort) combine structural and size-based signals into a single score. For AI cost, these are orthogonal: a function can be cheap to load (low AICP) but hard to reason about (high AIRD), or vice versa. Separating them gives the developer two actionable levers: reduce external calls to lower AICP, reduce cognitive complexity to lower AIRD.

4.2 What Existing Tools Miss

Beyond metrics, existing tools have deployment gaps relevant to AI-cost use cases.

Pre-commit integration. Of the 8 surveyed tools, only knots ships a pre-commit framework hook. The others require manual CI configuration or shell scripting. Pre-commit is the appropriate gate for per-

function complexity enforcement because it fires at the smallest unit of change (a commit), not at the coarse granularity of a CI pipeline run.

Multi-language support with per-function granularity. Most tools are either single-language (radon, pmccabe) or multi-language but file-granularity (tokei). Lizard offers multi-language per-function output but lacks Cognitive Complexity and AI-cost metrics.

Structured output for downstream tooling. SARIF output [?] is increasingly required for integration with GitHub Code Scanning and similar platforms. Of the surveyed tools, only SonarQube and knots produce SARIF; lizard and radon produce JSON but not SARIF.

4.3 Threshold Calibration

Threshold recommendations in the literature vary widely and are rarely empirically grounded. McCabe thresholds range from 10 (JPL [?]) to 25 (many style guides) to “no limit” (some Agile practitioners who argue test coverage is a better gate). No tool other than knots documents the corpus and distribution from which its default thresholds were derived.

The AIRD threshold of 85 was derived from the p99 distribution across 32,205 functions from 6 production codebases, then empirically validated against Sonnet 4.6 and Opus 4.8 behavior. This calibration methodology — corpus-derived ceilings with LLM-behavior validation — is a contribution independent of the specific metric, and one not found in prior tool documentation.

5 Related Work

Complexity metric surveys. Zuse [?] provides a comprehensive theoretical treatment of software complexity measures, covering axiomatic properties and measurement-theoretic validity. Shepperd [?] conducts an early empirical comparison of McCabe and Halstead against simpler size metrics, finding that SLOC predicts defect density as well as more sophisticated metrics. Gill and Kemerer [?] compare 8 metrics on a commercial COBOL codebase; they

find high inter-correlation, suggesting most metrics capture the same underlying construct.

Cognitive Complexity. Campbell [?] is the original specification. Gruner [?] provides a formal treatment of why Cognitive Complexity better predicts developer comprehension time than McCabe.

AI code generation quality. Studies of LLM code generation quality (e.g., Chen et al. [?] on Codex) evaluate correctness on benchmark problems but do not study how input function complexity affects LLM modification accuracy. This is the empirical gap motivating the companion paper’s AIRD/AICP validation work.

Technical debt metrics. SQALE [?] and the Maintainability Index [?] aggregate multiple metrics into a single remediation cost estimate. Neither was designed for LLM cost estimation, but the composite-metric approach informed AIRD’s design.

6 Conclusion

We have surveyed 10 complexity metrics and 8 open-source tools, characterizing where existing work falls short for AI-cost-aware complexity analysis. The gaps are consistent: no existing metric captures LLM context pressure or state access patterns; no tool other than knots ships both multi-language per-function analysis and pre-commit CI integration; and no tool documents empirically calibrated thresholds derived from a production codebase corpus.

The AIRD and AICP metrics described in the companion paper address the first gap directly. We release knots as an open-source tool to address the deployment gap, with a pre-commit hook, SARIF output, and documented threshold recommendations derived from a 32,205-function corpus.

References

- [1] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, 2(4):308–320, 1976.
- [2] M. H. Halstead. *Elements of Software Science*. Elsevier North-Holland, New York, 1977.
- [3] G. A. Campbell. Cognitive Complexity: A new way of measuring understandability. SonarSource SA, Geneva, Switzerland, 2018. <https://www.sonarsource.com/resources/cognitive-complexity/>.
- [4] B. A. Nejmeh. NPath: A measure of execution path complexity and its applications. *Communications of the ACM*, 31(2):188–200, 1988.
- [5] J. Fitzpatrick. Applying the ABC metric to C, C++ and Java. *C++ Report*, 9(9), 1997.
- [6] P. Oman and J. Hagemester. Metrics for assessing a software system’s maintainability. In *Proc. IEEE Conf. on Software Maintenance*, pages 337–344, 1992.
- [7] B. Raiter. pmccabe — McCabe cyclomatic complexity tool for C and C++. <https://people.debian.org/~bame/pmccabe/>, 1994.
- [8] T. Yin. Lizard: A code complexity analyzer. <https://github.com/terryyin/lizard>, 2013.
- [9] M. Barzoni. Radon: Code metrics for Python. <https://radon.readthedocs.io/>, 2013.
- [10] Mozilla. rust-code-analysis: A library to analyze and extract information from source code. <https://mozilla.github.io/rust-code-analysis/>, 2020.
- [11] X. Kipp. Tokei: A program that displays statistics about your code. <https://github.com/XAMPPRocky/tokei>, 2016.
- [12] SonarSource. SonarQube: Code quality and security. <https://www.sonarqube.org/>, 2007.
- [13] Scientific Toolkit. Understand: Static analysis and metrics. <https://www.scitools.com/>, 1993.
- [14] B. Arrendondo. knots: A multi-language code complexity analyzer with AI cost metrics. <https://github.com/brandon-arrendondo/knots>, 2024.

- [15] B. Arrendondo. AIRD and AICP: Metrics for predicting AI reasoning and context cost in code modification.
- [16] G. J. Holzmann. The Power of 10: Rules for Developing Safety-Critical Code. *IEEE Computer*, 39(6):95–97, 2006.
- [17] MISRA Consortium. MISRA C: 2012 — Guidelines for the Use of the C Language in Critical Systems, 2012.
- [18] OASIS SARIF Technical Committee. Static Analysis Results Interchange Format (SARIF) Version 2.1.0. OASIS Standard, 2020. <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html>.
- [19] H. Zuse. *A Framework of Software Measurement*. Walter de Gruyter, Berlin, 1998.
- [20] M. Shepperd. A critique of cyclomatic complexity as a software metric. *Software Engineering Journal*, 3(2):30–36, 1988.
- [21] G. K. Gill and C. F. Kemerer. Cyclomatic complexity density and software maintenance productivity. *IEEE Transactions on Software Engineering*, 17(12):1284–1288, 1991.
- [22] S. Gruner. On the conceptual difference between cognitive and cyclomatic complexity. *South African Computer Journal*, 54:53–71, 2014.
- [23] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [24] J.-L. Letouzey. The SQALE Method for Evaluating Technical Debt. In *Proc. 3rd Intl. Workshop on Managing Technical Debt*, pages 31–36, 2012.